

> Ficha Prática Nº6 (Jogo de Memória – Tabuleiro e Gestão do Top 10)

Esta ficha tem como objetivo implementar as funções necessárias para especificar **o nº de cartas do tabuleiro de acordo com o nível de jogo** e implementar o código necessário para apresentar, persistir e efetuar a gestão do **Top10**, isto é, pedir o nome ao utilizador quando a pontuação obtida permite entrar no top 10. Em relação ao nº de cartas do tabuleiro, pretende-se criar o tabuleiro de forma dinâmica, adaptando-se a qualquer nível de jogo da seguinte forma: 3 pares de cartas para o nível **básico**, 6 pares de cartas para o nível **intermédio** e 10 pares de cartas para o nível **avançado**.

O resultado da ficha apresenta-se na figura 1.

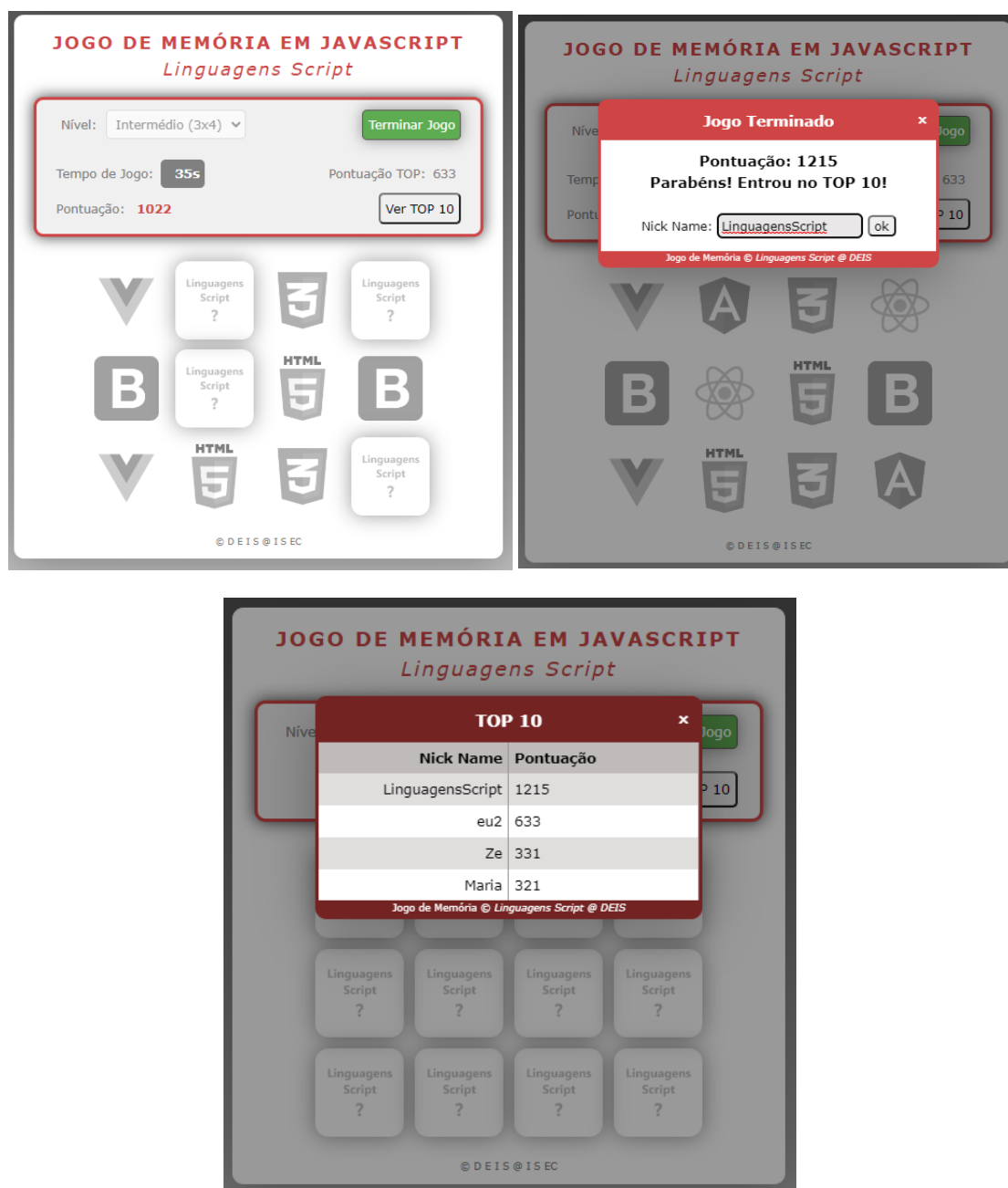


Figura 1 – Jogo de Memória em JavaScript – Imagens da aplicação – Top 10

> Preparação do ambiente

- a. Descompacte o ficheiro **fich6.zip**.

 **Os alunos que concluíram a resolução da ficha anterior, devem continuar nesse trabalho.** Os restantes alunos podem resolver esta ficha prática, tendo como base o código fornecido juntamente com esta ficha.

- b. Inicie o *Visual Studio Code* e abra a pasta no **workspace**. Visualize a página **index.html** no browser (recorra à extensão "*Live Server*"), no qual terá o aspeto da figura 2.

- c. Módulos a implementar:

- Gerar tabuleiro de acordo com o nível selecionado (Parte I);
- Apresentação do Top10 (Parte II);
- Gestão do Top 10 (Parte III);
- Persistir dados do Top10 com *localStorage* (Parte IV).



Figura 2 - Jogo (início)

Parte I – Criação de Tabuleiro Dinamicamente

Pretende-se especificar o código necessário para que o tabuleiro de jogo seja criado de forma dinâmica, isto é, que o número de cartas apresentadas no tabuleiro, dependa do nível selecionado. Portanto, quando o nível for **básico**, devem ser apresentadas 6 cartas, se o nível for **intermédio**, o tabuleiro deve ser composto por 12 cartas e, por fim, se avançado, o número de cartas deve ser 20. Podem ser usadas várias técnicas para criação do tabuleiro de jogo, no entanto, pretende-se implementar este módulo recorrendo aos métodos e/ou propriedades de manipulação do DOM.

- 1>** Especifique a função `createPanelGame` e invoque-a na função `reset`. Esta função irá criar o tabuleiro, de acordo com o nível selecionado. Para isso, implemente os seguintes passos.

- a. Elimine todas as peças existentes no tabuleiro **panelGame**. Este comportamento pode ser implementado com o código:

```
panelGame.innerHTML = '';
```

- b. Implemente o código necessário para gerar o código HTML de **uma** carta. Para isso recorra a vários métodos e/ou propriedades de manipulação do DOM, entre eles:

- `document.createElement(elemento)` - cria um *elemento*;
- `elemento.appendChild(elFilho)` - anexa o *elFilho* como último filho do *elemento*.
- `elemento.setAttribute(nome, valor);` - adiciona um novo atributo ou modifica o valor de um atributo já existente.

Por exemplo, o código abaixo, à esquerda, apresenta o *JavaScript* para gerar o HTML apresentado

JavaScript

```
let newDiv = document.createElement('div');
newDiv.setAttribute('class', 'card');
let imgBack = document.createElement('img');
imgBack.setAttribute('src', 'images/ls.png');
newDiv.appendChild(imgBack);
panelGame.appendChild(newDiv);
```

html

```
<div class='card'>
  <img src='images/ls.png'>
</div>
```

Assim, copie o código JS anterior e efetue as alterações necessárias, para completar a geração de uma carta do tabuleiro, que deve ter o seguinte código HTML:

```
<div class="card">
  
  <img class="card-front">
</div>
```

- c.** Duplica o **div** anteriormente criado, tantas vezes quantas necessárias, de acordo com o nível de jogo selecionado, **Básico** - 6 cartas | **Intermédio** - 12 cartas | **Avançado** – 20 cartas.

Esta duplicação de elementos pode ser efetuada com recurso ao `elemento.cloneNode(true)` que retorna a cópia de um elemento, incluindo todos os seus filhos, atributos e valores.

- d.** No fim da função, atualize a variável `cards` de acordo com o novo `panelGame`. Assim, deve especificar o seguinte código em que a propriedade `childNodes` permite obter todos os nodos de um elemento.

```
cards = panelGame.childNodes;
```

Nota: Certifique-se que a variável `cards` não é constante.

- e.** Adapte, na função `startGame`, o número de elementos no *array* `newCardLogos` para qualquer nível de jogo. Assim, substitua o limite de 3 por `cards.length/2`.
- f.** Confirme no browser o jogo com os vários níveis.
- g.** De forma a garantir uma distribuição diferente das cartas no tabuleiro (relativamente de nº de cartas por linha/coluna), dependendo do nível de jogo, adicione, na função `createPanelGame`, o código para aplicar as seguintes classes (já existentes no css) ao `panelGame`:

- > Se nível intermédio, adicionar classe `intermedio` (especifica *grid* 3x4)
- > Se nível avançado, adicionar classe `avancado` (especifica *grid* 4x5)

O nível básico, sendo o nível por omissão, já tem especificado uma *grid* de 2x3

- h. Para que qualquer das classes anteriores, seja removida do `panelGame`, sempre que se gera novo tabuleiro, aplique seguinte código (no início da função `createPanelGame`):

```
panelGame.className = '';
```

- i. Confirme no browser o jogo que deverá ter o aspeto das seguintes figuras.

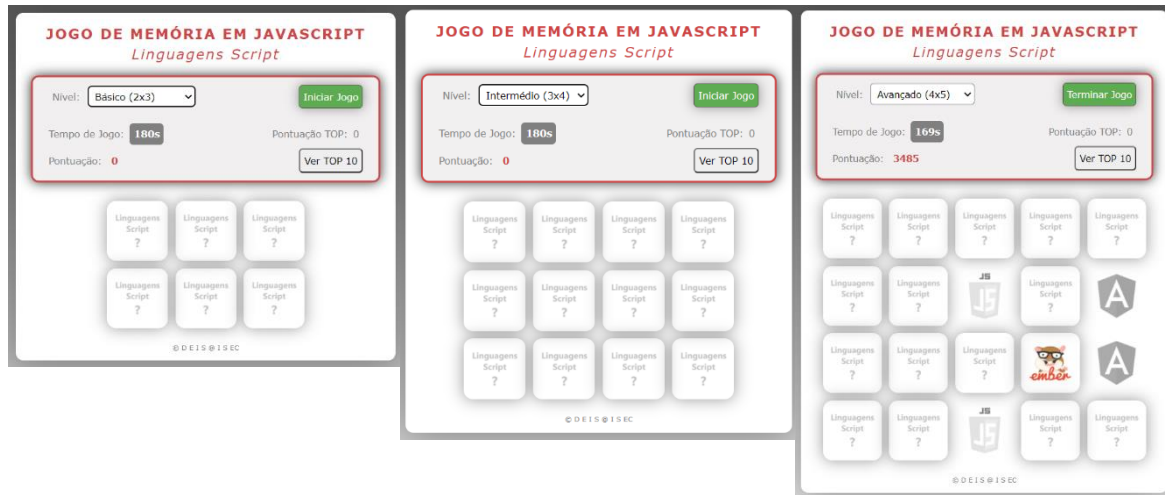


Figura 3 - Ficha 6

Parte II – Apresentação do TOP 10

- 2> Nesta fase, pretende-se apresentar a lista do TOP 10, ao clicar em “Ver TOP 10”, como se apresenta na figura 4.



Figura 4 - Top 10

- a. Para isso, declare o array `topGamers`, no *scope* global, e inicialize-o com dois objetos com duas propriedades, como se apresenta de seguida.

```
let topGamers = [
    { nickname: 'Ze', points: 331 },
    { nickname: 'Maria', points: 321 }
]
```

- b. Implemente a função `getTop10` que irá mostrados os dados do *array* na janela TOP 10. Para isso:
- Declare a variável `infoTop` que deve aceder ao elemento da página cujo id é `infoTop`, necessária para apresentar a lista dos jogadores.
 - Implemente um *ciclo* que percorra o array `topGamers` e construa uma *string* com os valores dos elementos existentes no array, no seguinte formato *nickName – points*.
 - Com recurso à variável `infoTop` e à propriedade `innerHTML`, escreva a *string* construída no painel do `top10`, como se apresenta a figura seguinte.



- Adicione um *event listener* ao **btTop** (não necessita de definir btTop pois já existe essa constante definida em *modal.js*), de forma a invocar a função **getTop10** quando há um click no botão TOP10.
- Confirme no **browser** o comportamento.

c. Como pode verificar, o aspeto da janela TOP 10 deve ser substancialmente melhorado. Para tal, implemente os seguintes passos:

- Comente o código implementado na alínea anterior, mais propriamente, a criação da *string* para apresentação dos jogadores no top10.
- De forma a impedir a visualização repetida do painel de resultados, sempre que a função é invocada deve limpar o conteúdo de infoTop, recorrendo para tal a:

```
infoTop.textContent = "";
```

- Crie elementos HTML, manipulando o DOM, de forma a obter a estrutura apresentada abaixo:

- > Como pode verificar, cada jogador (objeto) corresponde a um **bloco div com dois parágrafos**, um para o nickname e outro para a pontuação.

- > Substitua o código anterior de forma a criar estes elementos dinamicamente com recurso aos métodos e propriedades para manipulação do DOM como efetuado para criação do tabuleiro de jogo:

- o `createElement()`
- o `appendChild()`
- o `elemento.cloneNode(true)`
- o `textContent`
- o `firstChild/ lastChild`.

```
<div class="info" id="infoTop">
  <div>
    <p>Nick Name</p>
    <p>Pontuação</p>
  </div>
  <div>
    <p>Ze</p>
    <p>331</p>
  </div>
  <div>
    <p>Maria</p>
    <p>321</p>
  </div>
</div>
```

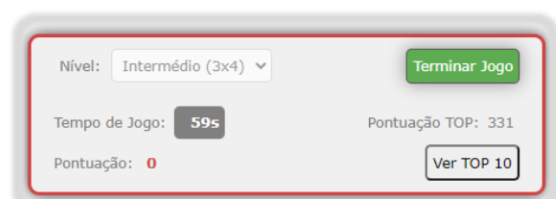
- Confirme no browser a visualização do TOP 10 que já deverá ter o aspeto desejado.

3> Pretende-se apresentar a “Pontuação TOP” quando se inicia o jogo, como apresentado na figura. Para isso:

a. Implemente a função **getTopPoints** que deve apenas obter os pontos do primeiro objeto existente no array e escrever os pontos elemento cujo ID é **pointsTop**.

b. Invoque a função anterior em **startGame**.

c. Confirme o resultado no browser.



Parte III – Gestão do TOP 10

- 4> Nesta fase, pretende-se verificar se o utilizador entra no TOP 10. Em caso afirmativo, é apresentado um campo para introdução do *nick name*, como apresentado na figura.



Para isso, implemente os seguintes passos:

- a. Implemente a função `getLastPoints`, no *scope global*, que deverá retornar apenas a pontuação do último jogador existente no *array* dos *topGamers*.
- b. Implemente a função `checkTop10` que verifica se o jogador entra ou não no top 10 e, se entrar, deve ser apresentada a caixa para especificar o seu **nickname**. Tenha em consideração:
 - Para um utilizador poder entrar no TOP10, deve garantir uma das condições: se ainda não existirem 10 jogadores na lista ou então, se existir, a pontuação é superior à última.
 - Recorra à função `getLastPoints` implementada anteriormente para obter o número de pontos obtidos pelo último utilizador.
 - A função `checkTop10` deve ser invocada na função `stopGame`
- c. Para mostrar a caixa de diálogo do nickname, deve:
 - Declarar a variável `nick` que permite aceder ao elemento da página cujo `id` é `nickname`. A visualização deve ser baseada na propriedade `display = 'block'`.
 - No elemento com `id` `messageGameOver`, por recurso à propriedade `innerHTML`, acrescentar ao conteúdo já existente nesse elemento, o texto "`<br>Parabéns! Entrou no TOP 10!`".
- d. Confirme o comportamento no browser.

- 5> Para gravar o nickname e os dados do utilizador, implemente os passos seguintes.

- a. Implemente a função `saveTop10`:
 - Criar um objeto com o nickname e pontuação. Para obter o *nickname*, deve aceder ao valor do elemento com `id` `inputNick`.
 - Verificar se o jogador já se encontra na lista do *topGamers*:
 - > Caso já exista, atualize a sua pontuação se a pontuação obtida no jogo for superior à existente no top.
 - > Se não existir, adicione o elemento no fim do *array*.
 - Ordenar o *array* através da pontuação dos elementos, da seguinte forma:

```
topGamers.sort(function (a, b) { return b.points - a.points });
```

- Se existirem mais de 10 elementos no *array*, eliminar o último.

- b.** Implemente o *eventListener* que invoque a função anterior, quando existir um click no botão cujo id é `okTop`. Para além de invocar a função `saveTop10`, também deve fechar a janela modal, recorrendo ao código `modalGameOver.close()`; e invoque a função `reset`.
- c.** Confirme o comportamento no browser e confirme que não existe erros na consola.

Parte IV – *localStorage*

Nesta secção, pretende-se que os dados do Top10 persistam, ainda que a janela do browser seja fechada. A propriedade *localStorage* do JavaScript permitir implementar esse comportamento, no qual os métodos `setItem` e `getItem` permitem armazenar e aceder a dados locais, respetivamente.

O exemplo abaixo apresenta uma forma de armazenar e aceder a dados, simples ou objetos.

```
localStorage.setItem('meuDado', '1234');
localStorage.getItem('meuDado')

localStorage.setItem('meuObjecto',
JSON.stringify({prop: '1', prop: '2'}));
meuObjecto = JSON.parse(meuObjecto)
```



Armazenar o Top10 localmente é algo que não faz sentido numa situação real (p.e. num jogo online, com vários jogadores), no entanto, é usado neste contexto para fins educacionais, de forma que o aluno possa exercitar o uso da propriedade *localStorage*.

- 6>** Para persistir os dados no jogo, implemente os seguintes passos:
- a.** Na função `saveTop10`, armazene os valores do *nickname* e a lista do top10, recorrendo ao método `setItem`;
 - b.** Implemente a função `getLocalStorage` que deverá obter os dados armazenados com o *localStorage*, nomeadamente o *nickname* e a lista do TOP10.
 - c.** Invoque a função anterior no *scope global*.
 - d.** Verifique no browser o comportamento o jogo, **que deverá estar concluído**, de acordo com o pretendido nas aulas LS.